Reto 8 – Observabilidad y Monitoreo del Ecosistema de Microservicios

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. En retos anteriores se construyó un ecosistema distribuido con:

- Despliegue con Docker y Docker Compose (Reto 1 y 2)
- Comunicación sincrónica y asincrónica (Retos 4)
- Seguridad con JWT (Reto 5)
- Automatización de pruebas (Reto 6)
- Integración Continua (Reto 7)

A pesar de todo esto, el sistema sigue siendo una "caja negra": cuando algo falla en producción, resulta extremadamente difícil saber **qué servicio falló, por qué, y qué petición lo provocó**. Si el Servicio de Empleados tarda 5 segundos en responder, ¿es culpa suya, de la base de datos, del broker de mensajes, o del servicio de departamentos al que llama?

En este reto se abordará uno de los pilares fundamentales de la arquitectura de microservicios en entornos reales: la **Observabilidad**. Se instrumentará el ecosistema para que responda a las tres preguntas clave de cualquier sistema distribuido:

- **¿Está vivo el sistema?** → *Health Checks*
- **¿Cómo se está comportando?** → *Métricas*
- **¿Qué hizo exactamente?** → *Trazabilidad Distribuida*
- **¿Alguien me avisa si algo falla?** → *Alertas*

Objetivo

Integrar un **stack de observabilidad** al ecosistema de microservicios existente. Al finalizar este reto, el sistema debe contar con una infraestructura de monitoreo que permita visualizar el estado de salud de cada servicio, analizar métricas de comportamiento en dashboards y rastrear visualmente el flujo de una petición a través de todos los microservicios involucrados. Adicionalmente, el sistema debe ser capaz de alertar proactivamente ante comportamientos anómalos.

Requisitos generales

La solución desarrollada debe cumplir con los siguientes requisitos:

- Cada microservicio debe exponer un **endpoint de salud** (`/health`) y un **endpoint de métricas** en formato compatible con Prometheus.
- Todos los microservicios deben estar **instrumentados con OpenTelemetry** para la trazabilidad distribuida, independientemente del lenguaje en que estén implementados.

- El ecosistema de observabilidad (Prometheus, Grafana, Zipkin/Jaeger) debe desplegarse como servicios adicionales en el `docker-compose.yml`.
- Debe construirse al menos **un dashboard en Grafana** que consolide métricas de múltiples microservicios.
- Deben configurarse **reglas de alerta** en Grafana que notifiquen a un canal externo ante comportamientos anómalos.
- La arquitectura de observabilidad debe estar **documentada** en el README.

1. Diseño de la Arquitectura de Observabilidad

Investigación previa

Antes de implementar, cada equipo debe comprender el flujo de telemetría y cómo se interconectan los componentes. El equipo debe investigar y documentar en el README los siguientes conceptos:

Concepto	Descripción a investigar
Las tres pilares de la Observabilidad	Métricas, Logs y Trazas. ¿Cuál es el rol de cada una?
Modelo Pull vs. Push	¿Cómo funciona el *scraping* de Prometheus (Pull)? ¿Cómo funciona el envío de trazas a Zipkin/Jaeger (Push)?
OpenTelemetry	¿Qué es la CNCF? ¿Por qué es relevante OpenTelemetry como estándar agnóstico?
W3C Trace Context	¿Cómo viaja el `traceId` a través de cabeceras HTTP entre servicios de distintos lenguajes?

Diagrama previo (Entregable)

Como primer entregable, cada equipo debe construir (antes de implementar) un diagrama de arquitectura que muestre:

1. Qué componentes del stack de observabilidad se van a levantar.
2. Cómo se conecta Prometheus a cada microservicio (¿a qué red de Docker pertenece?).
3. Cómo fluyen las trazas desde un microservicio hasta Zipkin/Jaeger.
4. Cómo Grafana consume datos de Prometheus.

> **Nota:** Este diagrama puede construirse con cualquier herramienta (draw.io, Mermaid, Excalidraw, etc.) y debe incluirse en el README o como imagen en el repositorio.

2. Infraestructura de Observabilidad (Docker Compose)

Stack de observabilidad de referencia

Este taller utiliza **Prometheus + Grafana + Loki + Zipkin/Jaeger** como stack de referencia por ser herramientas open-source, cloud-native y ampliamente adoptadas en la industria. Para el Proyecto Final, un equipo puede proponer alternativas (Datadog, New Relic, Grafana Cloud, ELK Stack, etc.) siempre que cubran los mismos requisitos funcionales y lo justifiquen en el README.

El equipo debe agregar los siguientes servicios al `docker-compose.yml` existente:

Servicio	Imagen de referencia	Puerto	Rol
Prometheus	`prom/prometheus:latest`	`9090`	Recolector de métricas (modelo Pull)
Grafana	`grafana/grafana:latest`	`3000`	Visualización de métricas, logs y gestión de alertas
Loki	`grafana/loki:latest`	`3100`	Agregación y almacenamiento de logs
Promtail	`grafana/promtail:latest`	—	Agente recolector de logs (envía a Loki)
Zipkin *(o Jaeger)*	`openzipkin/zipkin:latest`	`9411`	Servidor de trazabilidad distribuida

> **Elección:** El equipo puede elegir entre **Zipkin** o **Jaeger** como servidor de trazas. Debe documentar en el README cuál eligió y por qué.

Requisitos de configuración

- Prometheus, Grafana y el servidor de trazas deben estar en la **misma red Docker** que los microservicios.
- Grafana debe configurarse con **Prometheus como fuente de datos** (*datasource*) de forma automatizada al levantar el contenedor (usando archivos de aprovisionamiento en volúmenes).
- Los datos de Grafana (dashboards, configuraciones) deben **persistir en un volumen** para no perderse al reiniciar.

Ejemplo de estructura de servicios en `docker-compose.yml`

```
services:
  # ... servicios existentes (empleados, departamentos, auth, etc.) ...

  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./observability/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    networks:
      - microservices-network

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana
      - ./observability/grafana/provisioning:/etc/grafana/provisioning
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    networks:
      - microservices-network

  zipkin:
    image: openzipkin/zipkin:latest
    ports:
      - "9411:9411"
    networks:
      - microservices-network

volumes:
  grafana-data:
```

> **Importante:** Este fragmento es solo una referencia estructural. La configuración completa (redes, dependencias, variables de entorno) debe ser implementada y completada por el equipo.

3. Configuración de Prometheus

Prometheus necesita saber a qué servicios debe "raspar" (*scrape*) para recolectar métricas. Esta configuración se define en el archivo `prometheus.yml`.

Requisitos del archivo `prometheus.yml`

- Configurar al menos **un** `scrape_job` por cada microservicio del ecosistema.
- Utilizar los **nombres de red de Docker** (los nombres de los servicios definidos en `docker-compose.yml`) como targets, no IPs fijas.
- El puerto y la ruta del endpoint de métricas de cada microservicio puede variar según el lenguaje y el framework utilizado.

Ejemplo de estructura del archivo

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'api-gateway'
    metrics_path: '/metrics'   # Ajustar según el framework/lenguaje del servicio
    static_configs:
      - targets: ['api-gateway:8080']

  - job_name: 'empleados-service'
    metrics_path: '/metrics'   # Ajustar según el framework/lenguaje del servicio
    static_configs:
      - targets: ['empleados-service:8081']

  - job_name: 'departamentos-service'
    metrics_path: '/metrics'   # Ajustar según el framework/lenguaje del servicio
    static_configs:
      - targets: ['departamentos-service:8082']

  - job_name: 'auth-service'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['auth-service:8083']

  - job_name: 'perfiles-service'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['perfiles-service:8084']

  - job_name: 'notificaciones-service'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['notificaciones-service:8085']

  - job_name: 'vacaciones-service'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['vacaciones-service:8086']
```

> **Son siete objetivos, no dos.** El ecosistema a esta altura tiene el API Gateway (Reto 3), los servicios de empleados y departamentos (Reto 2), los de perfiles, notificaciones y vacaciones (Reto 4) y el de autenticación (Reto 5). Si algún servicio falta en `prometheus.yml`, su caída pasará desapercibida — que es exactamente el problema que este reto viene a resolver.
>
> Ajuste los puertos a los que realmente usa su ecosistema.
>
> **Importante:** El **API Gateway debe ser el primer `scrape_job` configurado**. Al ser el punto de entrada único del sistema, sus métricas de latencia y tasa de error ofrecen la visión más valiosa del estado del sistema desde la perspectiva del cliente.
>
> **Importante:** La ruta del endpoint de métricas (`metrics_path`) puede variar según el lenguaje: - **Spring Boot (Java):** `/actuator/prometheus` - **Express (Node.js):** `/metrics` (usando `prom-client`)

> - **FastAPI (Python):** `/metrics` (usando `prometheus-fastapi-instrumentator`) - **Go:** `/metrics` (usando `prometheus/client_golang`)
>
> Cada equipo debe identificar y documentar la ruta correcta según el lenguaje de cada uno de sus servicios.

4. Instrumentación de los Microservicios

4.1 Exposición de Métricas (Formato Prometheus)

Independientemente del lenguaje utilizado, cada microservicio debe exponer un endpoint que retorne métricas en **texto plano en formato Prometheus**. Este es el estándar de la industria.

A continuación se listan las librerías de referencia más comunes. El equipo debe investigar e integrar la correspondiente a cada lenguaje utilizado:

Lenguaje / Framework	Librería de referencia	Endpoint típico
Java / Spring Boot	`micrometer-registry-prometheus` (vía Spring Boot Actuator)	`/actuator/prometheus`
Node.js / Express	`prom-client`	`/metrics`
Python / FastAPI	`prometheus-fastapi-instrumentator`	`/metrics`
Python / Flask	`prometheus-flask-exporter`	`/metrics`
Go	`prometheus/client_golang`	`/metrics`
.NET / ASP.NET Core	`prometheus-net.AspNetCore`	`/metrics`
Otro lenguaje	Buscar en prometheus.io/docs/instrumenting/clientlibs	Variable

Las métricas mínimas que deben estar disponibles por defecto son: - **Uso de CPU y memoria** del proceso. - **Número de peticiones HTTP** recibidas (separadas por código de respuesta). - **Latencia de las peticiones HTTP** (tiempo de respuesta).

4.2 Trazabilidad Distribuida con OpenTelemetry

OpenTelemetry (OTel) es el estándar abierto de la CNCF para la instrumentación de sistemas distribuidos. Permite que un `traceId` generado en el primer servicio de una cadena de llamadas se propague automáticamente a través de todos los servicios involucrados, independientemente del lenguaje en que estén escritos.

Objetivo de la trazabilidad

Al finalizar este punto, debe ser posible abrir la UI de Zipkin/Jaeger, buscar un `traceId`, y visualizar una **cascada de spans** que muestre: 1. Qué servicio recibió la petición original. 2. A qué otros servicios llamó (y cuánto tardó cada llamada). 3. Si algún servicio retornó un error.

Librerías de referencia por lenguaje

Lenguaje	SDK / Librería de referencia
Java / Spring Boot	`opentelemetry-spring-boot-starter` + exportador OTLP/Zipkin
Node.js	`@opentelemetry/sdk-node` + `@opentelemetry/exporter-zipkin`
Python	`opentelemetry-sdk` + `opentelemetry-exporter-zipkin`
Go	`go.opentelemetry.io/otel` + exportador Zipkin
.NET	`OpenTelemetry.Exporter.Zipkin`
Otro	Buscar en opentelemetry.io/docs/languages

Configuración mínima esperada

Cada microservicio debe configurar: - **Nombre del servicio** (`service.name`): Debe ser descriptivo (ej. `empleados-service`). Esto aparecerá en la UI de Zipkin/Jaeger. - **Exportador**: Apuntando a la URL del servidor de Zipkin/Jaeger desplegado en Docker Compose. - **Propagación de contexto**: Configurada con el estándar **W3C Trace Context** para asegurar compatibilidad entre lenguajes.

Ejemplo de variables de entorno de referencia (aplicables a varios SDKs de OTel):

```
OTEL_SERVICE_NAME=empleados-service
OTEL_EXPORTER_ZIPKIN_ENDPOINT=http://zipkin:9411/api/v2/spans
OTEL_PROPAGATORS=tracecontext,baggage
```

> **Importante:** La URL del exportador debe usar el **nombre de red de Docker** (`zipkin`), no `localhost`.

El caso difícil: trazas sin petición que las origine

La instrumentación automática funciona porque hay una **petición HTTP entrante** que crea la traza raíz. Pero el `vacaciones-service` tiene un flujo que no empieza así: el **scheduler** dispara `vacaciones.iniciadas` sin que nadie haya llamado a nada. No hay petición, no hay traza padre, y la cadena que sigue — `auth-service` suspende la cuenta, `notificaciones-service` envía el correo — aparece en Zipkin como spans sueltos, sin relación entre sí.

Es justo el flujo que más cuesta depurar cuando algo falla, porque ocurre de madrugada y sin nadie mirando.

Lo que debe hacer: en la tarea programada, **iniciar un span raíz manualmente** antes de publicar el evento, y propagar su contexto en las cabeceras del mensaje.

```
# Ejemplo conceptual (Python / OTel) — el patrón es el mismo en todos los SDK
with tracer.start_as_current_span("scheduler.vacaciones.iniciar") as span:
    span.set_attribute("vacaciones.id", periodo.id)
    span.set_attribute("empleado.id", periodo.empleado_id)

    # Inyectar el contexto de traza en las cabeceras del mensaje
    headers = {}
    TraceContextTextMapPropagator().inject(headers)
    broker.publish("vacaciones.iniciadas", evento, headers=headers)
```

Y en el consumidor, **extraer** ese contexto para que su span cuelgue del mismo árbol en lugar de empezar uno nuevo.

> **Por qué importa más de lo que parece:** la propagación por HTTP la hace el SDK sola, pero **la propagación a través del broker casi siempre hay que hacerla a mano**. Sin esto, la mitad asíncrona del sistema —que es donde vive toda la lógica de este ecosistema— queda fuera de las trazas.
>
> **Verificación:** en Zipkin, la traza del inicio de vacaciones debe mostrar una sola cascada que vaya del scheduler al `auth-service` y de ahí al `notificaciones-service`. Si ve tres trazas independientes, el contexto no se está propagando.

4.3 Endpoint de Salud (`/health`)

Cada microservicio debe exponer un endpoint `GET /health` que retorne un JSON indicando su estado operativo.

Respuesta esperada (servicio saludable):

```
{
  "status": "UP",
  "service": "empleados-service",
  "checks": {
    "database": "UP",
    "messageBroker": "UP"
  }
}
```

Respuesta esperada (servicio degradado):

```
{
  "status": "DOWN",
  "service": "empleados-service",
  "checks": {
    "database": "DOWN",
    "messageBroker": "UP"
  }
}
```

El endpoint debe retornar HTTP `200` cuando el estado es `UP` y HTTP `503 Service Unavailable` cuando el estado es `DOWN`.

> Para servicios en **Spring Boot**, esta funcionalidad está integrada en `/actuator/health`. Para otros lenguajes, debe implementarse manualmente verificando la conectividad con la base de datos y el broker.

4.4 Logs Estructurados (Loki + Promtail)

Los **logs** son el tercer pilar de la observabilidad y son un requisito del Proyecto Final. Un log **estructurado** (en formato JSON) es infinitamente más útil que un log de texto plano porque puede filtrarse, agregarse y correlacionarse con las trazas.

¿Por qué Loki?

Loki es el sistema de agregación de logs de Grafana Labs. A diferencia del ELK Stack (Elasticsearch + Logstash + Kibana), es más liviano y se integra nativamente en el mismo Grafana que el equipo ya tiene configurado, agregando una fuente de datos adicional sin levantar más herramientas.

Configuración mínima de Promtail

Promtail es el agente que recolecta los logs de los contenedores Docker y los envía a Loki. Se configura con un archivo `promtail-config.yml`:

```
# observability/promtail/promtail-config.yml
server:
  http_listen_port: 9080

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: docker-containers
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 5s
    relabel_configs:
      - source_labels: ['__meta_docker_container_name']
        target_label: 'container'
      - source_labels: ['__meta_docker_container_label_com_docker_compose_service']
        target_label: 'service'
```

> **Importante:** El servicio `promtail` en el `docker-compose.yml` necesita acceso al socket de Docker (`/var/run/docker.sock`) como volumen para poder descubrir los contenedores automáticamente.

Logs estructurados por lenguaje

Para que los logs sean útiles en Loki, cada microservicio debe emitir logs en **formato JSON**. A continuación las librerías de referencia:

Lenguaje / Framework	Librería de referencia	Configuración clave
Java / Spring Boot	Logback + `logstash-logback-encoder`	Agregar appender JSON en `logback-spring.xml`
Node.js	`winston` con `winston-json-formatter`	Configurar `format: winston.format.json()`
Python	`structlog` o `python-json-logger`	Configurar `JSONRenderer` como procesador final
Go	`zap` o `zerolog`	Ambas emiten JSON por defecto
.NET	`Serilog` con `Serilog.Sinks.Console` en JSON	Configurar `outputTemplate` en formato JSON

Campos mínimos que debe incluir cada log

```
{
  "timestamp": "2026-04-30T15:00:00Z",
  "level": "INFO",
  "service": "empleados-service",
  "traceId": "abc123def456",
  "message": "Empleado creado exitosamente",
  "employeeId": "E010"
}
```

> **Nota:** El campo `traceId` es clave: permite **correlacionar un log con su traza distribuida** en Zipkin/Jaeger. Los SDKs de OpenTelemetry configurados en la sección 4.2 inyectan este campo automáticamente en el contexto de logging de la mayoría de los frameworks.

5. Dashboard en Grafana

Grafana se conecta a Prometheus como fuente de datos y permite construir visualizaciones a partir de las métricas recolectadas.

Requisitos del Dashboard

Cada equipo debe construir un dashboard en Grafana que incluya **al menos** los siguientes paneles:

Panel	Métrica a visualizar	Tipo de gráfica sugerida
Estado de cada servicio	`up{job="nombre-servicio"}`	*Stat* (verde/rojo)

Panel	Métrica a visualizar	Tipo de gráfica sugerida
Tasa de peticiones por servicio	`rate(http_requests_total[1m])`	*Time Series*
Latencia promedio	`rate(http_request_duration_seconds_sum[1m]) / rate(http_request_duration_seconds_count[1m])`	*Time Series*
Errores HTTP (4xx y 5xx)	`sum by (status) (rate(http_requests_total{status=~"[45].."}[1m]))`	*Time Series*

> **Nota:** Los nombres exactos de las métricas (`http_requests_total`, `http_request_duration_seconds_sum`, etc.) pueden variar según la librería de instrumentación utilizada. Consultar la documentación de la librería correspondiente. El equipo debe adaptar las consultas PromQL según las métricas reales que exponga cada servicio.

Organización sugerida del Dashboard

El dashboard debe tener mínimo dos **filas (rows)**: 1. **Resumen del Sistema:** Estado de salud de todos los servicios (paneles tipo *Stat*). 2. **Comportamiento del Tráfico:** Métricas de peticiones y latencia (gráficas de serie temporal).

6. Alertas Proactivas

Un sistema de monitoreo reactivo (donde un humano mira el dashboard) no es suficiente en sistemas de producción. Las alertas hacen que el sistema sea **proactivo**: notifica automáticamente al equipo de soporte cuando algo falla o se comporta de forma anómala.

Configuración de Alertas en Grafana

Grafana permite definir reglas de alerta directamente desde la interfaz de usuario, sin necesidad de herramientas adicionales.

Reglas de alerta requeridas

Cada equipo debe configurar **al menos dos** de las siguientes reglas de alerta:

#	Regla	Condición	Descripción
1	**Servicio Caído**	`up{job="<nombre-servicio>"} == 0` durante 1 minuto	Un microservicio dejó de responder al scraping de Prometheus.
2	**Alta Tasa de Errores**	Porcentaje de respuestas HTTP 5xx > 10% durante 2 minutos	El servicio está retornando errores de servidor de forma sostenida.
3	**Alta Latencia**	Latencia promedio de respuesta > 2 segundos durante 2 minutos	El servicio está respondiendo lentamente, posiblemente por sobrecarga o dependencia lenta.
4	**Contenedor Reiniciado**	Métrica de reinicios del contenedor aumenta	Un servicio colapsó y fue reiniciado por Docker.

Canal de notificación (*Contact Point*)

El canal de notificación es el destino al que Grafana enviará la alerta cuando se dispare la regla. Se debe configurar **al menos uno** de los siguientes canales:

Canal	Complejidad	Observación
Webhook de Discord	Baja	Crear un servidor de Discord y generar un Webhook. Grafana lo soporta nativamente.
Bot de Telegram	Baja	Crear un bot con `@BotFather` y configurar el `chat_id`. Grafana lo soporta nativamente.
Slack	Media	Requiere crear una app de Slack y configurar un Incoming Webhook.
Email	Media	Requiere configurar un servidor SMTP en Grafana (se puede usar Mailhog en Docker para pruebas).

> **Documentar en el README:** Qué canal de notificación eligió el equipo y cómo se configuró.

7. Pruebas del Sistema: Simulación del Caos

Verificar el funcionamiento completo del stack de observabilidad provocando situaciones anómalas de forma controlada.

Flujo de prueba sugerido

1. **Levantar el ecosistema completo:** `bash docker-compose up --build`

2. **Verificar que el stack de observabilidad está activo:** - Prometheus UI: `http://localhost:9090` → sección *Targets* debe mostrar todos los microservicios con estado `UP`. - Grafana: `http://localhost:3000` → el dashboard debe mostrar métricas en tiempo real. - Zipkin/Jaeger: `http://localhost:9411` (Zipkin) o `http://localhost:16686` (Jaeger).

3. **Generar tráfico para poblar las métricas:** ```bash # Crear un empleado (genera trazas y métricas) curl -X POST http://localhost:/empleados \ -H "Content-Type: application/json" \ -H "Authorization: Bearer " \ -d '{"id": "E010", "nombre": "Ana Gómez", "email": "ana@empresa.com", "departamentoId": "IT", "fechaIngreso": "2026-04-01"}'

Realizar múltiples consultas para generar métricas de tráfico for i in {1..10}; do curl http://localhost:/empleados -H "Authorization: Bearer "; done ```

4. **Verificar la traza distribuida:** Abrir Zipkin/Jaeger, buscar las trazas recientes y localizar la traza correspondiente a la creación del empleado. **La traza debe mostrar la cascada completa:** desde el gateway (si existe) → empleados-service → departamentos-service → message broker → auth-service → notificaciones-service y perfiles-service.

5. **Simulación de Caos – Apagar un servicio:** `bash # Detener el contenedor de un microservicio (sin eliminarlo) docker-compose stop departamentos-service` - Esperar ~2 minutos y verificar:

    - En **Prometheus**: el target de `departamentos-service` pasa a estado `DOWN`.
    - En **Grafana**: el panel de estado del servicio cambia a rojo.
    - En el **canal de alertas** (Discord/Telegram/etc.): debe llegar la notificación de "Servicio Caído".

6. **Simulación de Caos – Inducir errores:** Introducir temporalmente en el código de un servicio una excepción o retardo artificial: `python # Ejemplo en Python import time, random if random.random() < 0.5: time.sleep(5) # Latencia artificial` Rebuilding el contenedor y verificar en **Grafana** que el panel de latencia refleja el incremento. Si configuraron la alerta de "Alta Latencia", debe dispararse.

7. **Documentar los hallazgos:** En la sección de pruebas del README, incluir: - Captura de pantalla del Dashboard de Grafana con métricas reales. - Captura de pantalla de la vista de trazas en Zipkin/Jaeger mostrando la cascada completa. - Captura de pantalla de la alerta recibida en el canal de notificación (Discord/Telegram/etc.). - Responder: "*¿Qué servicio del ecosistema tardó más en responder y cómo lo identificaron?*"

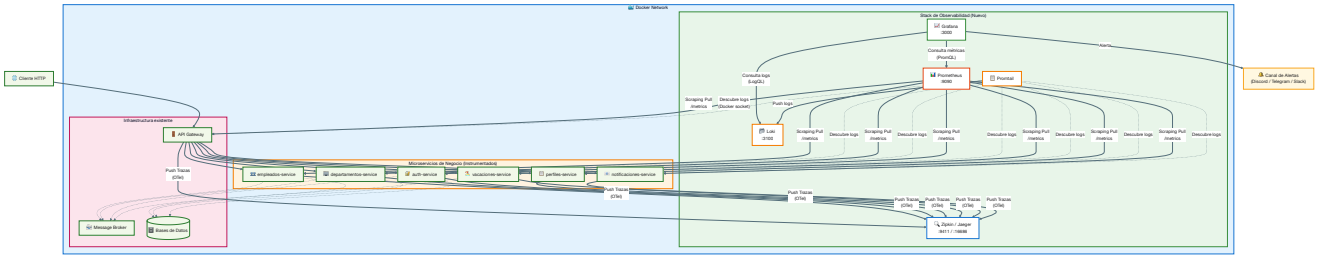
Entregables

- Código versionado en sus respectivos repositorios de GitHub.
- Archivos de configuración de observabilidad en una carpeta `observability/` en el repositorio:
- `observability/prometheus/prometheus.yml`
- `observability/grafana/provisioning/` (datasources y dashboards exportados en JSON)
- `docker-compose.yml` actualizado con los servicios de observabilidad.
- README.md actualizado que incluya:
- Diagrama de arquitectura de observabilidad (diseñado en el paso 1).
- Investigación de los conceptos solicitados (Pull vs. Push, OTel, W3C Trace Context).
- Documentación de las librerías utilizadas en cada microservicio para métricas y trazabilidad.
- Justificación de la elección entre Zipkin y Jaeger.
- Documentación del canal de alertas elegido y cómo configurarlo.
- Capturas de pantalla de las pruebas de caos (dashboard, trazas y alerta recibida).
- Respuesta a la pregunta: "*¿Qué servicio del ecosistema tardó más en responder y cómo lo identificaron?*"

Consideraciones

- Todos los servicios del ecosistema deben estar instrumentados, no solo los nuevos.
- Las URLs de los servicios de observabilidad dentro de Docker Compose deben usar los **nombres de red de Docker**, no `localhost`.
- Los archivos de configuración de Grafana (datasources, dashboards) deben ser versionados en Git para que el dashboard esté disponible automáticamente al hacer `docker-compose up`, sin configuración manual.
- La "simulación del caos" es **obligatoria** y debe estar evidenciada con capturas de pantalla en el README.

Diagrama de Arquitectura Esperada



Criterios de Evaluación

El taller se evalúa sobre **5 puntos**, distribuidos de la siguiente forma:

#	Elemento	Valor	Aspectos a evaluar
1	**Infraestructura de Observabilidad**	1.0	Prometheus, Grafana, Loki/Promtail y Zipkin/Jaeger correctamente configurados en `docker-compose.yml`. Correcta configuración de redes de Docker para que Prometheus alcance todos los microservicios (incluido el API Gateway). Archivo `prometheus.yml` con todos los servicios como targets.
2	**Instrumentación de Microservicios**	1.0	Todos los microservicios (incluido el API Gateway) exponen métricas en formato Prometheus y el endpoint `/health`. Las librerías OpenTelemetry están correctamente configuradas en cada servicio con el nombre de servicio, exportador y propagación W3C configurados. **La traza del onboarding se ve completa de extremo a extremo** en la UI de Zipkin/Jaeger.
3	**Logs Centralizados**	1.0	Todos los microservicios emiten logs estructurados (formato JSON). Loki y Promtail correctamente configurados para recolectar logs de todos los contenedores. Grafana configurado con Loki como fuente de datos adicional. Al menos una consulta LogQL evidenciada en el README.
4	**Dashboard en Grafana**	0.5	Dashboard con al menos los 4 paneles requeridos (estado, tasa de peticiones, latencia, errores). El dashboard está aprovisionado automáticamente (exportado en JSON y versionado en Git).
5	**Alertas Proactivas**	0.5	Al menos dos reglas de alerta configuradas en Grafana. Canal de notificación externo (Discord/Telegram/Slack/Email) funcional y documentado. Evidencia (captura de pantalla) de una alerta recibida en el canal.
6	**Pruebas de Caos y Documentación**	1.0	Simulación de caos documentada con capturas de pantalla (dashboard, traza distribuida y alerta). Diagrama de arquitectura de observabilidad en el README. Respuesta fundamentada a la pregunta de análisis.

> **Nota:** La documentación (README.md) y el versionamiento adecuado en GitHub son requisitos transversales que afectan la evaluación de cada elemento.